Hybrid Routing 도입 시점 + 위험 분석 + 안전 구현 전략

작성: 2026-05-08 · JM Tech One — Voice AI Pilot **범위**: 현재 Pilot 단계 진단 + Hybrid 최적 시점 + 즉시 도입 위험 + 5-step 안전 구현 + 2-모델 안정성/확장성

임원 요약 (Executive Summary)

결론	핵심
현재 Pilot 위치	M1.4 (1매장 라이브 검증, silent agent 진단 직후, retry fix 적용 직전)
5매장 deploy 상태	DB에 4 매장 active이나 **voice agent 라우팅은 1매장만** (Twilio 번호 1개)
Hybrid 즉시 도입 위험	코드 변경 ~250 LOC × 보호 구역 인접 × baseline 측정 부재 = **위험** ★★★★
권장 Hybrid 시점	**M6-M7 (5매장 안정 운영 + 1주 KPI 데이터 후 ≈ 4-6주 후)**
즉시 가능한 안전 액션	(1) retry fix (2) Tier 2 (3) usage 캡처 read-only (Step A)
5-Step 안전 구현 총 기간	8-10주 (Step A 1주 + B 1주 + C 1주 + D 2-3주 + E 2-3주)

핵심 한 줄: **Hybrid는 옳은 방향이지만 잘못된 시점**. 1매장도 안정 안 된 상태에서 2-모델 분기는 측정 노이즈 + 위험 누적. **5매장 baseline 확보 후 incremental rollout**이 risk-adjusted 최적.

Part 1 — 현재 Pilot 단계 진단

마일스톤 매핑 (M0 → M9)

Milestone	상태	완료 시점	비고
M0: 코드 안정화 (Wave A.3)	✅	2026-05-08 (오전)	latency 3200→293ms, email 100%
M1.1: Wave A.3 라이브 검증	✅	2026-05-08 09:30	1매장 통화
M1.2: CRM Wave 1 라이브 검증	✅	2026-05-08 10:00	Welcome back + auto-fill 작동
M1.3: Silent agent 진단	✅	2026-05-08 11:00	TPM rate_limit_exceeded 확정
M1.4: Retry fix 적용	🟡 진행 중	(이번 주)	**현재 위치**
M1.5: Tier 2 도달 ($50 누적)	⏸️	(~7일)	$50 충전 + 7일
M1.6: 1매장 24-48h 무인 운영	⏸️	(~1주)	안정성 검증
M2: 1매장 KPI baseline 보고서	⏸️	(~2주)	AHT, completed_rate, returning%
M3: 5매장 deployment	⏸️	(~2-3주)	Twilio 번호 4개 추가 + store 라우팅
M4: 5매장 안정 운영 검증	⏸️	(~3-4주)	7일 무인 운영
M5: 5매장 KPI 보고서 + Series A pre-prep	⏸️	(~4-5주)	펀드 deck 1차
M6: A/B test 인프라 구축	⏸️	(~5-6주)	DB 컬럼 + 라우팅 함수 + 모니터링
M7: 2.0 canary 1매장	⏸️	(~6-7주)	단일 모델 비교
M8: ⭐ Hybrid Routing 가동	⏸️	**(~8-10주)**	mini + 2.0 자동 분기
M9: Vertical 확장 (KBBQ/Sushi/Mexican)	⏸️	(~12-16주)	다국어 매장

현재 운영 실태 (객관 데이터)

```
- 활성 매장 (DB):        4개 (JM Home Services, JM Beauty Salon, JM Auto Repair, JM Cafe)
- Voice agent 라우팅:     1개 (JM Cafe → +1-503-994-1265)
- 최근 7일 통화:         52건 (~7-8건/일, Pilot 검증 강도)
- 누적 commits (branch): 25+ (Wave A.3 + CRM Wave 1 + diag 강화)
- 활성 코드 size:         realtime_voice.py 1,152 LOC + recital.py 141 + flows.py 1,812
- Production-ready 보호 구역: 16개 (Wave A.3 16개 항목)
```

Pilot 진행률 시각화

```
M0 ── M1.1 ── M1.2 ── M1.3 ── M1.4 ── M1.5 ── M1.6 ── M2 ── M3 ── M4 ── M5 ── M6 ──
✅     ✅      ✅      ✅      🟡      ⏸️      ⏸️      ⏸️   ⏸️   ⏸️   ⏸️   ⏸️
                               ↑ 현재
```

진행률: 4/16 마일스톤 완료 = **25%**.

Part 2 — Hybrid 도입 최적 시점 분석

시점 선택의 트레이드오프

시점	장점	위험	정량 평가
즉시 (M1.4-M1.5)	단가 일찍 절감 ($550/월)	1매장 안정 미검증, baseline 부재, 보호 구역 충돌	Risk ★★★★ Reward ★★★
M3 (5매장 deploy 후)	단일 모델 baseline 확보	5매장 운영 미검증, A/B 인프라 없음	Risk ★★★ Reward ★★★
M6-M7 (5매장 안정 + 1주 후) ⭐	baseline + 인프라 + 운영 안정 모두 갖춤	4-6주 지연	**Risk ★★ Reward ★★★★★**
M8+ (KPI 보고 + 펀드 deck 후)	펀드 narrative에 single-model + hybrid 비교 데이터	6-8주 지연	Risk ★ Reward ★★★★

왜 M6-M7이 최적인가 (정량 분석)

A. Baseline 데이터 확보 측면

- **즉시**: 1매장 7일 데이터 = 52 calls (통계적 의미 없음, σ 큼)
- **M5**: 5매장 14일 = 약 1,800-2,100 calls (충분, completed_rate 5pp delta 검출 가능)
- **M6-M7**: 5매장 21+일 = 2,500+ calls (robust)

B. 코드 회귀 위험 측면

- **즉시**: Wave A.3 보호 구역 16개 fresh + CRM Wave 1 fresh + Hybrid 코드 신규 = **3 layer 동시 변경**
- **M5**: Wave A.3 + CRM Wave 1 4-5주 stable + Hybrid 신규 = 1 layer만 신규
- **M6-M7**: 모든 이전 layer stable + Hybrid 인프라 ready = **회귀 위험 1/3**

C. 펀드 어필 측면 (Series A 데이터)

- **즉시**: "Pilot 시작과 동시에 Hybrid 시도, 측정 노이즈 큼"
- **M5 single-model 데이터**: "5매장 1.5 단일로 X 단가 검증, 시장 평균 대비 Y× 마진"
- **M8 hybrid 데이터**: "M5 baseline 대비 Hybrid로 -30% 추가 절감 입증" — **데이터 기반 narrative 가장 강력**

D. TPM 헤드룸 측면

- **즉시**: Tier 1 (40K). 두 모델 모두 같은 quota. Hybrid 효과 부분 상쇄
- **M1.5 후**: Tier 2 (200K). 두 모델 분리 안전 가능
- **M6+**: Tier 3 (800K) 가능. Hybrid scale 자유

E. A/B test 통계 유의성

- **즉시**: 7-8 calls/일 × 50% mini split = 3-4 mini calls/일. 1,090 calls/arm 도달 = **300+ 일** 필요 → 의미 없음
- **5매장**: 150 calls/일 × 50% = 75 mini calls/일 → 1,090 도달 14-15일 (적정)
- **vertical 확장 후**: 더 빠름

결론 — Hybrid 최적 시점

M6-M7 (5매장 안정 운영 + 1주 KPI 보고서 작성 후)

근거: 1. **baseline 확보**: 5매장 단일 모델 운영으로 단가 + 품질 baseline 확정 2. **회귀 위험 최소**: Wave A.3 + CRM Wave 1 모두 4-5주 stable 3. **펀드 narrative robust**: "A/B로 30% 절감 입증" = 데이터 기반 4. **A/B 통계 유의성**: 14-15일 만에 결정 가능 5. **TPM 안전**: Tier 3 도달 후 Hybrid 분기 자유

예상 일정: - 오늘 (M1.4) → M5 5매장 KPI 보고서: **4-6주** - M5 → M6 인프라 구축: **1주** - M6 → M7 2.0 canary 단일: **1주** - M7 → M8 Hybrid 가동: **2-3주** - **총 8-10주 후 Hybrid 가동** (2026년 7월 초~중순 예상)

Part 3 — 즉시 Hybrid 시 위험 분석

코드 변경 인벤토리 (즉시 도입 시)

파일	변경 유형	LOC 추정	위험도
`backend/scripts/migrate_ab_test_columns.sql`	신규	~30	★★ (NULLABLE 안전, 그러나 production migration)
`backend/app/api/realtime_voice.py`	수정	~80	★★★★ (보호 구역 인접: session.create, response.done, session_state)
`backend/app/services/bridge/transactions.py`	수정	~40	★★ (token 필드 추가)
`backend/app/core/config.py`	수정	~30	★ (settings 추가)
`backend/.env`	수정	~10	★ (env var)
`backend/app/services/store_config.py` (신규?)	신규	~60	★★ (override 캐시 필요)
`tests/unit/api/test_ab_assignment.py`	신규	~150	★ (테스트 자체)
`tests/unit/services/test_token_capture.py`	신규	~80	★
합계		**~480 LOC**	**★★★★ 종합**

핵심 위험 시나리오 (예상)

🚨 Risk-1: realtime_voice.py 회귀

무엇: session.create + response.done 핸들러는 Wave A.3 16개 보호 구역 중 다수와 인접. 모델 분기 로직 추가 시 미세 회귀 가능. **확률**: ★★★ (3/5) **영향**: silent agent 패턴 다른 형태로 재현 / NATO recital reconcile 깨짐 / `[perf] call_end` 측정 부정확

🚨 Risk-2: 모델별 API 응답 형식 미세 차이

무엇: gpt-realtime-1.5 / 2.0 / mini의 `response.done.usage` 필드 구조가 정확히 동일한지 미검증. mini는 voice 풀 공유 + 8-tool 처리에서 응답 형식 차이 가능. **확률**: ★★★ (3/5) **영향**: token 캡처 NULL → 비용 reconcile 부정확, 진단 로그 누락

🚨 Risk-3: 통계적 무의미

무엇: 1매장 7-8 calls/일 × 50% split = 3-4 mini calls/일. 1,090 calls/arm 도달 = 300+ 일. **확률**: ★★★★★ (5/5 — 확정적) **영향**: A/B 데이터 자체가 noise — 펀드 어필 narrative 약함

🚨 Risk-4: 모델 voice 미세 차이

무엇: Marin이 1.5/2.0/mini에서 prosody (억양, 속도) 미세 차이. 통화 중 모델 변경은 없지만 매장별 일관성 깨짐. **확률**: ★★ (2/5) **영향**: caller가 "오늘은 목소리가 좀..." 인지 가능 / 매장 staff 피드백 혼란

🚨 Risk-5: mini의 KO/JA/ZH 회귀

무엇: 다국어 매장 미적용해도 첫 turn STT가 잘못 detect 시 mini로 라우팅 → 회귀. **확률**: ★★★ (3/5) **영향**: KO/JA/ZH 통화 fallback 로직 필수 → 추가 코드 위험

🚨 Risk-6: 측정 도구 부재로 비교 불가

무엇: 현재 model_variant, token_usage 컬럼 없음. Hybrid 도입 후 "효과 입증" 데이터 없음. **확률**: ★★★★ (4/5) **영향**: Series A에서 "Hybrid를 왜 했나?" 질문에 데이터 답변 불가

🚨 Risk-7: rate_limit retry 미적용 상태

무엇: 현재 silent agent fix (retry) 미적용. mini도 같은 TPM 공유 → mini 사용해도 rate limit 직격. **확률**: ★★★★ (4/5) **영향**: Hybrid 효과 측정 자체가 noise

🚨 Risk-8: store_configs override 캐시 미스

무엇: 매번 DB 조회 시 추가 latency. cache 안 만들면 통화 시작 +50-100ms. **확률**: ★★ (2/5) **영향**: latency 회귀 (Wave A.3 노력과 충돌)

종합 위험 평가

시점	Risk Score (5×요소)	Reward Score
즉시 도입	**30/40** (★★★★)	8/20 (시간 단축만)
M3 (5매장 후)	18/40 (★★★)	12/20
M6-M7 (안정 + baseline 후)	**8/40 (★★)**	**18/20**
M8+ (KPI 보고 후)	6/40 (★)	16/20 (시간 손실)

Risk-Adjusted Reward = (Reward / Risk): - 즉시: 8/30 = **0.27** - M3: 12/18 = 0.67 - **M6-M7: 18/8 = 2.25** ← 최적 - M8+: 16/6 = 2.67 (지연 페널티 제외 시)

Part 4 — 5-Step 안전 구현 전략

각 단계는 **이전 단계가 안정 검증되어야** 다음으로 진입. 회귀 시 즉시 직전 단계로 복귀.

Step A — Read-only Instrumentation (1주, 위험 ★)

목표: 코드 흐름 변경 0, 데이터 수집만 시작.

변경: 1. `bridge_transactions` 에 `model_id` , `tokens_*` 5개 컬럼 추가 (NULLABLE) 2. `realtime_voice.py` `response.done` 핸들러: `usage` 필드 캡처만 (분기 X) 3. `update_call_metrics` 에 token 필드 추가 4. 1매장에서 그대로 운영, 매 통화에 model_id="gpt-realtime-1.5" 자동 기록

측정: 7일 후 SQL로 통화당 평균 토큰 사용량 reconcile (이론치 vs 실측)

Gate to Step B: 토큰 reconcile 오차 ≤ 10%, 회귀 0건

Rollback: 컬럼 NULLABLE이라 두어도 무해. 코드 revert 1 commit.

Step B — 2.0 Single-Model Canary (1주, 위험 ★★)

목표: 모델 자체 차이 측정 (단일 vs 단일).

변경: 1. `OPENAI_REALTIME_MODEL` env var = `gpt-realtime-2` 2. `reasoning.effort: "low"` session config 추가 3. 1매장 (JM Cafe) 만 적용. 매장 분기 X.

측정: AHT, TTFA, completed_rate, tool_call_failures vs 1.5 baseline

Gate to Step C: AHT not worse +10s, tool_failures not worse +1pp, voice quality 동일

Rollback: `OPENAI_REALTIME_MODEL=gpt-realtime-1.5` env var 1줄 (즉시).

Step C — Manual Override Hybrid (1주, 위험 ★★)

목표: 매장별 수동 모델 설정 (자동 라우팅 X).

변경: 1. `store_configs.realtime_model_override` 컬럼 추가 (TEXT NULLABLE) 2. `_select_model(store)` 함수 — override 있으면 사용, 없으면 env default 3. `realtime_voice.py session.create` 모델 결정에 사용 4. 영업/설치 담당이 vertical별 수동 매핑: - JM Cafe → "mini" (영어 단순 흐름) - (KBBQ/Sushi 매장 추가 시) → "gpt-realtime-2"

측정: 매장별 단가 비교 (model_id로 분리 집계)

Gate to Step D: 5매장 24-48시간 무회귀, model_id별 KPI 데이터 일치

Rollback: store_configs 한 row UPDATE로 즉시 복귀.

Step D — Hash-based Auto Routing (2-3주, 위험 ★★★)

목표: 매장 내에서 % split 자동 분기.

변경: 1. `AB_MINI_PCT` env var (default 0) 2. `_select_model()` 에 hash(call_sid) % 100 < AB_MINI_PCT 로직 추가 3. canary 10% → 25% → 50% 단계적 ramp 4. 회로 차단기: 10통 연속 fail → 30분간 매장 default로 복귀

측정: 동일 매장 내 mini vs full A/B (페어드 데이터)

Gate to Step E: completed_rate 동일 ±5pp, tool_failures 동일 ±1pp, voice 만족도 동일

Rollback: `AB_MINI_PCT=0` 즉시.

Step E — Language + Complexity Detection (2-3주, 위험 ★★★)

목표: 첫 turn STT로 언어/복잡도 판정 → 자동 분기.

변경: 1. 첫 turn `conversation.item.input_audio_transcription.completed` 이벤트에서 language 판정 2. 복잡 모디파이어 (≥3개) 감지 → 2.0로 boost 3. mid-stream 모델 변경 X (call_sid 결정성 유지) 4. 매장별 base default × 통화별 detection 조합

측정: 언어별 라우팅 정확도 (KO 통화가 mini로 잘못 가는 비율 추적)

Gate to Phase 4: 언어별 routing 정확도 ≥ 95%, 회귀 0건

Rollback: `LANG_DETECT_ENABLED=false` env var.

5-Step 일정표

Step	기간	누적 시간	위험 누적	가치
A. Read-only	1주	1주	★	토큰 데이터 인프라
B. 2.0 canary	1주	2주	★★	모델 비교 baseline
C. Manual override	1주	3주	★★	vertical 분기 시작
D. Hash auto	2-3주	5-6주	★★★	A/B 통계 검출
E. Lang detect	2-3주	7-9주	★★★	full Hybrid 가동

총 7-9주 (M6-M7 시점부터 시작 → M8 가동) = 위 Pilot 마일스톤과 정확히 일치.

Part 5 — 2-모델 시스템 안정성 + 확장성 검토

시스템 안정성 검토

1. Voice 일관성

우려: caller가 통화마다 미세하게 다른 prosody 경험 (mini vs full Marin). **현실**: Marin은 양 모델에서 동일 voice 풀 공유 (community 보고). 발성 데이터셋 동일. **미세 차이**: full은 GPT-5-class 추론 → 더 풍부한 표현; mini는 단순/빠름. **대응**: Step B에서 사람 청취 평가 5건으로 검증. 차이 인지 시 매장별 모델 고정. **평가**: ✅ **안정성 영향 미미** (관리 가능).

2. Tool calling reliability

우려: mini는 8 tools 처리에서 정확도 88-92% (community 보고). 우리 8 tools 환경에서 ~10% 호출 실패 가능. **현실**: 실패 시 retry + manual_alert tier3 fallback 가능 (이미 인프라 있음). **대응**: Step C에서 model_variant별 tool_call_failures 모니터링 + 회로 차단기 (실패 ≥10% → full 강제). **평가**: ⚠️ **mini는 단순 흐름만** (allergen Q&A, recall_order). 복잡 multi-step은 full 강제.

3. TPM 한계 공유

우려: full + mini 모두 같은 OpenAI 계정 quota 공유. Hybrid 도입한다고 TPM 한계 안 늘어남. **현실**: 단, mini는 비싸지 않으니 같은 통화 처리 시 토큰 절반 → quota 효율 ↑. **대응**: Tier 2/3 도달 후 Hybrid 진입. retry fix 필수. **평가**: ⚠️ **Tier 상승 + retry fix 선행 필수**.

4. API 응답 형식 차이

우려: gpt-realtime-2.0의 `response.done.usage` 구조가 1.5와 정확 동일한지 미검증. mini는 더더욱. **현실**: OpenAI Realtime API 사양상 동일 형식 보장. 그러나 reasoning_tokens 등 신규 필드 가능. **대응**: Step A read-only로 모델별 응답 dump 7일 수집 → 차이 발견 시 fix. **평가**: ✅ **Step A에서 사전 확인 가능**.

5. 모델 가용성

우려: mini가 갑자기 down / deprecated / latency spike 가능. **현실**: gpt-realtime-mini는 GA. 1.5는 곧 deprecation 예측. 2.0은 최신. **대응**: 회로 차단기 + fallback chain (mini fail → 2.0 fail → 1.5). **평가**: ⚠️ **fallback chain 필수**.

6. Voice 응답 cancel/interrupt 동작 차이

우려: server_vad/silence_duration_ms가 모델별 동작 미세 차이 가능. **현실**: VAD는 OpenAI 서버 측 — 모델 무관 동일. **대응**: Step B canary로 검증. **평가**: ✅ **모델 무관**.

7. CRM Wave 1 호환성

우려: customer_context block 길이 6,500 tokens가 mini context window에 부담. **현실**: mini context = 32K, 우리 prompt = ~6.5K + history. 여유 있음. **대응**: customer_context block은 visit_count==0 시 미주입 (이미 구현). 여유 충분. **평가**: ✅ **현재 설계 호환**.

확장성 검토

1. 라우팅 latency

우려: store_configs DB 조회 + hash 계산 시 매 통화 +5-100ms. **대응**: - store_configs는 in-memory cache (10분 TTL) — 1회 lookup → 모든 통화 재사용 - hash 계산 = SHA1 단일 콜 < 1ms **평가**: ✅ **5ms 이내** 달성 가능.

2. 모니터링 복잡도

우려: 모델별 분리 추적 (per-model AHT, completed_rate, cost) — 대시보드 복잡 ↑. **대응**: model_id 컬럼 + group by SQL. 기존 KPI 쿼리에 GROUP BY 1줄 추가. **평가**: ✅ **단일 컬럼 추가로 해결**.

3. 펀드 어필 KPI 일관성

우려: "분당 단가" 발표 시 어떤 숫자? 모델별 가중 평균? **대응**: KPI 보고서에 "Hybrid 가중 평균: $X / Single-model 1.5: $Y / mini-only: $Z" 3-way 표기. **평가**: ✅ **투명 보고로 해결**.

4. 매장 수 확장 (5 → 50 → 500)

우려: vertical별 모델 매핑 룰이 매장 수 증가에 따라 복잡. **대응**: store_configs.realtime_model_override = 매장 row level 설정. 자동 default + override만 관리. **평가**: ✅ **scalable**.

5. 모델 라인업 변경 (gpt-realtime-3.0 등 미래)

우려: OpenAI 신규 모델 출시 시 라우팅 룰 재설계. **대응**: env var + store_configs override 패턴은 모델 추상화. 신규 모델 추가는 env var 1줄. **평가**: ✅ **future-proof**.

6. Vertical 확장 (KBBQ → Sushi → Mexican → Chinese)

우려: 각 vertical 다른 modifier 처리, 다른 언어 → 모델 선택 룰 분기. **대응**: vertical column이 store_configs에 이미 있음. (vertical, language) → model 매핑 표. **평가**: ✅ **인프라 준비됨**.

7. 회로 차단기 + fallback

우려: mini 실패 시 자동 full 복귀 룰 복잡. **대응**: 매장별 sliding window (직전 20 통화 중 3+ fail → full 강제 30분). **평가**: ⚠️ **Step D에서 필수 구현**.

종합 안정성·확장성 평가

항목	안정성	확장성	전제 조건
Voice 일관성	✅	✅	Step B 검증
Tool reliability	⚠️	✅	mini 단순 흐름만
TPM 한계	⚠️	✅	Tier 상승 + retry fix 선행
API 응답 형식	✅	✅	Step A 사전 확인
모델 가용성	⚠️	✅	fallback chain
VAD 동작	✅	✅	모델 무관
CRM Wave 1 호환	✅	✅	현재 설계 안전
라우팅 latency	✅	✅	in-memory cache
모니터링	✅	✅	model_id 컬럼만
KPI 일관성	✅	✅	3-way 보고
매장 확장	✅	✅	store_configs override
모델 라인업 변경	✅	✅	env var 추상화
Vertical 확장	✅	✅	(vertical, lang) 매핑
Fallback chain	⚠️	✅	Step D 회로 차단기

결론: 4개 ⚠️ 항목 모두 **Step A-E에서 자연스럽게 해결됨**. 즉시 도입 시 4개 동시 위험 → 5-step 분산 시 단계별 1-2개씩 해결.

Part 6 — 권장 + 의사결정 게이트

최종 권장 — 순차 진행, M6-M7 Hybrid

즉시 (이번 주, M1.4 → M1.5)

1. ✅ rate_limit_exceeded retry fix 코드 commit
2. ✅ OpenAI $50 충전 → 7일 후 Tier 2 자동
3. ✅ Live 통화 검증 (1매장)

1-2주 (M1.6 → M2)

4. 1매장 24-48시간 무인 운영
5. KPI baseline 측정 (AHT, completed_rate, returning_rate)

3-4주 (M3 → M4)

6. 5매장 deployment (Twilio 번호 4개 추가, store 라우팅 매핑)
7. 5매장 안정 운영 검증 (1주)

5-6주 (M5 → M6)

8. 5매장 KPI 보고서 + Series A pre-prep deck 1차
9. **Step A: 토큰 instrumentation** (1주, 위험 ★)

7-10주 (M7 → M8)

10. **Step B: 2.0 canary** (1주, 위험 ★★)
11. **Step C: Manual override Hybrid** (1주, 위험 ★★)
12. **Step D: Hash auto routing** (2-3주, 위험 ★★★)
13. **Step E: Language + complexity detection** (2-3주, 위험 ★★★)

의사결정 게이트 (Each Step)

각 step 진입 전 **명시적 사용자 승인** + 데이터 게이트 통과 필수:

Step	진입 조건	회귀 시 액션
A	5매장 7일 안정	모니터링만 — 회귀 ≈ 0
B	A 토큰 reconcile ≤10% 오차	env var 1줄 복귀
C	B AHT/voice 동등	store_configs UPDATE
D	C 5매장 24-48h 안정	AB_MINI_PCT=0
E	D completed_rate 동등 ±5pp	LANG_DETECT_ENABLED=false

절대 하지 말 것

Anti-pattern	이유
즉시 Hybrid 도입	1매장 안정 미검증 + baseline 부재 + 회귀 위험 ★★★★
Step 건너뛰기	각 step의 측정/게이트 의미 무력화
Multi-step 동시	회귀 분기 디버깅 불가
Tier 상승 안 하고 Hybrid	mini도 같은 TPM 공유 — 효과 측정 오염
Retry fix 없이 Hybrid	silent agent 발생 시 어느 모델 탓인지 불명
5매장 deploy 전 A/B	통계 무의미 (300+일 필요)

Risks / Unknowns

항목	영향	검증 방법
Pilot 일정 가정 (4-6주 5매장 deploy)	Hybrid 시점 전체 shift	매주 review
5매장 vertical 분포 가정	mini route % 변동	M3 deploy 시 확정
통화 평균 시간/turn 수 변화	per-call cost 가정 부정확	usage 캡처 후 reconcile
OpenAI 1.5 deprecation timeline	M3-M5 사이 강제 migration 가능	OpenAI 발표 모니터링
2.0 production 안정성 (1일 미만)	Step B에서 회귀 가능	Step B canary로 격리
mini의 KO/JA/ZH 실측 품질	50-utterance/lang eval 미실시	Step C 직전 1일 eval sprint
Series A 일정 압박	Hybrid 단축 압박 가능	데이터 우선 narrative — 압박 시 M5 baseline만으로 단가 우위 입증 가능

부록 A — 즉시 도입 시 코드 변경 미리보기 (참고용)

Step A-E를 모두 거치지 않고 즉시 적용 시 필요한 코드 변경 (참고용 — 권장 X)

```
-- migration_ab_test.sql
ALTER TABLE bridge_transactions
  ADD COLUMN IF NOT EXISTS model_id text,
  ADD COLUMN IF NOT EXISTS model_variant text,
  ADD COLUMN IF NOT EXISTS tokens_audio_in int,
  ADD COLUMN IF NOT EXISTS tokens_audio_out int,
  ADD COLUMN IF NOT EXISTS tokens_text_in_fresh int,
  ADD COLUMN IF NOT EXISTS tokens_text_in_cached int,
  ADD COLUMN IF NOT EXISTS tokens_text_out int;

ALTER TABLE store_configs
  ADD COLUMN IF NOT EXISTS realtime_model_override text;
```

```
# realtime_voice.py — _select_model 함수
import hashlib

def _select_model(call_sid: str, store_config: dict) -> tuple[str, str]:
    """Per-call model selection.
    Priority: store override > env hash split > env default.
    """
    override = (store_config.get("realtime_model_override") or "").lower()
    if override:
        return MODEL_MAP[override], override

    ab_mini_pct = int(os.getenv("AB_MINI_PCT", "0"))
    if ab_mini_pct > 0:
        h = int(hashlib.sha1(call_sid.encode()).hexdigest()[:8], 16) % 100
        if h < ab_mini_pct:
            return settings.openai_realtime_model_mini, "mini"
    return settings.openai_realtime_model_full, "full"
```

```
# response.done 핸들러 — usage 캡처
elif etype == "response.done":
    response = getattr(event, "response", None)
    usage = getattr(response, "usage", None) or {}
    if usage:
        session_state["tokens_audio_in"]      += usage.get("input_audio_tokens", 0)
        session_state["tokens_audio_out"]     += usage.get("output_audio_tokens", 0)
        session_state["tokens_text_in_fresh"] += usage.get("input_text_tokens", 0)
        session_state["tokens_text_in_cached"] += usage.get("input_cached_tokens", 0)
        session_state["tokens_text_out"]      += usage.get("output_text_tokens", 0)
    # ... 기존 진단 로그 유지
```

부록 B — 권장 5-Step 코드 변경 분포

Step	신규 LOC	수정 LOC	테스트 LOC	위험도
A. Read-only	30 (SQL) + 30 (transactions)	30 (response.done)	50	★
B. 2.0 canary	0	5 (env var)	0	★★
C. Manual override	60 (store_config) + 20 (_select_model)	20 (session.create)	80	★★
D. Hash auto	30 (회로 차단기)	10 (_select_model)	100	★★★
E. Lang detect	80 (detector)	30 (router)	120	★★★
합계	**220**	**95**	**350**	**★★★ 분산**

즉시 도입 시 위험 ★★★★ (~480 LOC 동시 변경, 다 새 코드) vs **5-step 위험 ★★★ 분산**.

출처 (Sources)

1. JM Tech One — 라이브 운영 데이터 (commit chain `8390f62` 시점, 2026-05-08)
2. JM Tech One — 마이그레이션 status memory (`project_migration_status.md`)
3. OpenAI Realtime API Docs (3-way 모델 비교 보고서 출처 참조)

4. Wave A.3 보호 구역 정의 (`session_resume_2026-05-08_wave-a3.md`)
5. CRM Wave 1 spec (`docs/superpowers/specs/2026-05-08-crm-wave-1-design.md`)

End of Report.

Distribution: founder + technical advisor materials. Next review: M2 5매장 deploy 시점에 일정 갱신.